UNIVERSIDAD COOPERATIVA DE COLOMBIA
Campus Pasto, Nariño
Diseño de Software · Cuarto semestre

Taller práctico Bloque 7

SmartLibrary — Caso de uso «Registrar préstamo»

Del modelo de clases orientado a objetos a un diseño por tres capas para una API REST con Spring Boot

Presentado por
Cristhian Santiago Parra Erazo
ID 926222

Docente
Miguel Tovar

Repositorio: github.com/santhiagoxinge/TallerPractico7-SmartLibrary
Pasto, noviembre de 2026

Contenido

1. Caso de estudio y alcance 3
2. Actividad 1 · Leer y cuestionar el modelo 4
2.1 ¿Qué datos y operaciones son propios del dominio? 4
2.2 ¿Por qué Libro–Prestamo y Estudiante–Prestamo son asociaciones y no composiciones? 4
2.3 Operaciones necesarias para consultar, validar y guardar 4
2.4 Responsabilidades que faltan 4
Diagrama 01 · Modelo inicial anotado 5
3. Actividad 2 · Asignar responsabilidades 6
4. Actividad 3 · Modelo de clases por capas 7
Diagrama 02 · Clases por capas 8
4.1 Justificación de relaciones y multiplicidades 9
5. Actividad 4 · Secuencia del caso de uso 10
Diagrama 03 · Secuencia POST /api/prestamos 11
5.1 Verificación: cada mensaje corresponde a una operación del diagrama de clases 12
6. Actividad 5 · Preparación de la implementación 13
7. Actividad 6 · Implementación y pruebas 13
8. Actividad 7 · Revisión y defensa 14
8.1 ¿Qué responsabilidades cambiaron del modelo inicial al modelo por capas? 14
8.2 ¿Qué patrón GRASP se reconoce en la coordinación? ¿Por qué un controlador GRASP no equivale necesariamente a @RestController? 14
8.3 ¿Dónde se ubica la regla de disponibilidad y por qué? 14
8.4 ¿Qué relaciones UML conservaron y cuáles modificaron? 14
8.5 ¿Cómo se comprueba la trazabilidad requisito → clase → operación → prueba? 14
9. Decisiones de diseño 16

Si el índice aparece vacío, en Word: clic derecho sobre él → «Actualizar campos».

1. Caso de estudio y alcance

SmartLibrary gestiona libros, estudiantes y préstamos. Este taller diseña e implementa el caso de uso **«Registrar préstamo»** (`POST /api/prestamos`) y la **consulta de préstamos** (`GET /api/prestamos`). Los estudiantes y libros ya están registrados; la persistencia se hace con **H2** en memoria mediante **Spring Data JPA**.

Reglas de negocio del taller, que se usan como requisitos para la trazabilidad:

Id	Regla
R1	Solo un libro disponible puede prestarse.
R2	El préstamo registra estudiante, libro y fecha.
R3	Al prestarlo, el libro deja de estar disponible.
R4	Una solicitud inválida no debe crear préstamo.
R5	Un libro puede tener varios préstamos históricos, pero solo uno activo a la vez.

Alcance. Se sigue el modelo didáctico del taller (Estudiante, Libro, Prestamo). Las reglas de la guía general del proyecto que no pide este taller (ejemplares, máximo tres préstamos activos por usuario, fecha límite de devolución) quedan fuera del alcance y se mencionan como evolución futura.

Artefactos. Cada diagrama se elaboró en Visual Paradigm 18.1 (Community Edition), en un proyecto `.vpp` independiente, y se exportó como imagen. Todo el material está en el repositorio:

- `diagramas/vpp/` — proyectos de Visual Paradigm (01, 02, 03).
- `diagramas/img/` — imágenes exportadas que aparecen en este documento.
- `smartlibrary-api/` — proyecto Spring Boot (Java 21, Maven).
- `evidencias/` — salida de las pruebas automáticas y de las peticiones HTTP.

2. Actividad 1 · Leer y cuestionar el modelo

Se reconstruyó el modelo inicial del enunciado sin cambios (cuatro clases, cinco asociaciones con multiplicidad 1 a 0..*) y se le añadieron cuatro notas con las observaciones (diagrama 01, página siguiente).

2.1 ¿Qué datos y operaciones son propios del dominio?

- **Estudiante:** `id`, `nombre`, `correo` y sus métodos de acceso.
- **Libro:** `id`, `titulo`, `disponible`; `isDisponible()` y `marcarPrestado()` son comportamiento del dominio porque expresan R1 y R3.
- **Prestamo:** `id`, `fechaPrestamo`, `fechaDevolucion [0..1]` y las referencias a `estudiante` y `libro` (R2). Que `fechaDevolucion` esté vacía indica que el préstamo está activo (R5).
- **GestorPrestamos no es dominio.** Mezcla almacenamiento en listas con la coordinación del caso de uso; por eso desaparece en el rediseño.

2.2 ¿Por qué Libro–Prestamo y Estudiante–Prestamo son asociaciones y no composiciones?

Una composición exige una relación todo-parte en la que la parte no puede existir sin el todo y muere con él. Aquí no se cumple: el estudiante y el libro existen antes del préstamo y siguen existiendo después; incluso conservan un historial de varios préstamos. El préstamo solo **referencia** a un estudiante y a un libro, y sus ciclos de vida son independientes. Por eso se usa una asociación con multiplicidad **1** en el lado de Estudiante o Libro y **0..*** en el lado de Prestamo. Tampoco se justifica la agregación, porque no hay semántica todo-parte.

2.3 Operaciones necesarias para consultar, validar y guardar

- **Consultar:** buscar estudiante por id, buscar libro por id y listar préstamos.
- **Validar:** comprobar que la solicitud trae ambos ids (R4), que el estudiante y el libro existen, que el libro está disponible (R1) y que no tiene un préstamo activo (R5).
- **Guardar:** guardar el libro con su nueva disponibilidad (R3) y guardar el préstamo (R2) en una misma transacción.

2.4 Responsabilidades que faltan

- **Para HTTP:** recibir `POST` y `GET`, convertir el JSON en un objeto de entrada, validar el formato y traducir el resultado a códigos 201, 200, 400, 404 y 409.
- **Para datos:** `findById`, `save`, `findAll` y una consulta de préstamo activo por libro. En el modelo inicial esto se resolvía con listas en memoria dentro de GestorPrestamos.
- **Otras:** `registrarPrestamo(e, l)` recibe objetos, pero la petición HTTP solo trae identificadores, y no existe un mecanismo para informar errores (404 y 409).

Diagrama 01 · Modelo inicial anotado

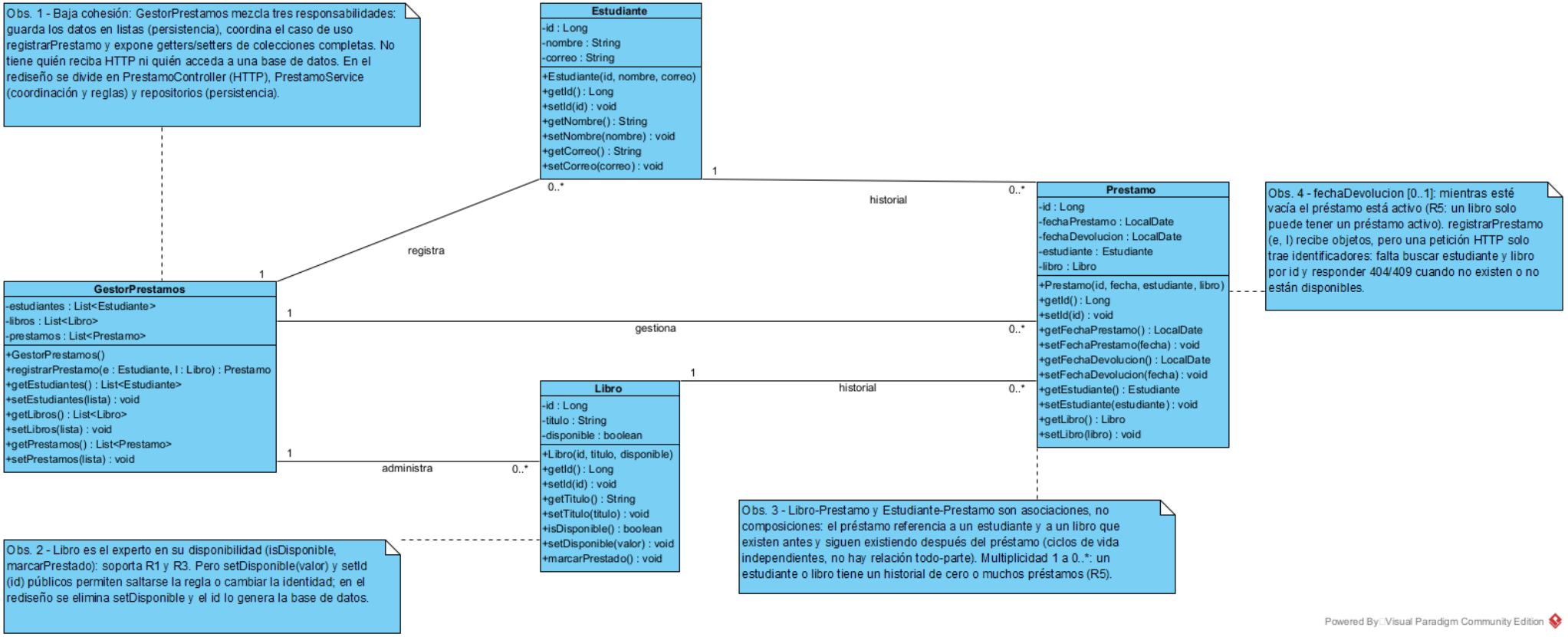


Figura 1. Modelo inicial del enunciado con las observaciones de la Actividad 1 (diagramas/vpp/01-modelo-inicial-anotado.vpp).

3. Actividad 2 · Asignar responsabilidades

Las entidades Estudiante, Libro y Prestamo se mantienen como modelo del dominio y no forman una cuarta capa de ejecución.

Operación o decisión	Clase responsable	Justificación
Recibir POST /api/prestamos	PrestamoController `registrarPrestamo(request)`	Capa de presentación: solo entrada y salida HTTP. Valida el formato con `@Valid` y delega; no contiene reglas de negocio.
Buscar estudiante por identificador	EstudianteRepository `findById(id)`, invocado por PrestamoService	Acceso a datos aislado. El Service decide qué hacer si no existe (404).
Buscar libro por identificador	LibroRepository `findById(id)`, invocado por PrestamoService	Mismo criterio: el repositorio solo consulta; la decisión es del Service.
Validar disponibilidad	PrestamoService, apoyado en `Libro.isDisponible()` y `PrestamoRepository.existsByLibroIdAndFechaDevolucionIsNull()`	Regla del caso de uso (R1, R5). Libro es el experto en su estado; el Service la combina con los préstamos activos.
Crear el préstamo	PrestamoService `new Prestamo(estudiante, libro, fecha)`	GRASP Creador: el Service tiene los datos que inicializan el préstamo (R2).
Cambiar disponibilidad	Libro `marcarPrestado()`	GRASP Experto: el libro cambia su propio estado (R3). Se elimina `setDisponible()` para que no se pueda saltar la regla.
Guardar cambios	LibroRepository `save(libro)` y PrestamoRepository `save(prestamo)`, coordinados por PrestamoService con `@Transactional`	La persistencia queda en los repositorios. La transacción garantiza que el cambio de disponibilidad y el préstamo se guarden juntos o ninguno (R3, R4).
Devolver respuesta HTTP	PrestamoController (201 + Location, 200) y ManejadorExcepciones (404, 409, 400)	La presentación traduce resultados y excepciones a códigos de estado; el Service no conoce HTTP.

4. Actividad 3 · Modelo de clases por capas

El diagrama 02 organiza las clases en los mismos paquetes del código: `controller`, `service`, `repository`, `model`, `dto` y `exception`. Las dependencias siguen una sola dirección: **Controller** → **Service** → **Repository**. El Controller nunca usa un Repository.

Diagrama 02 · Clases por capas

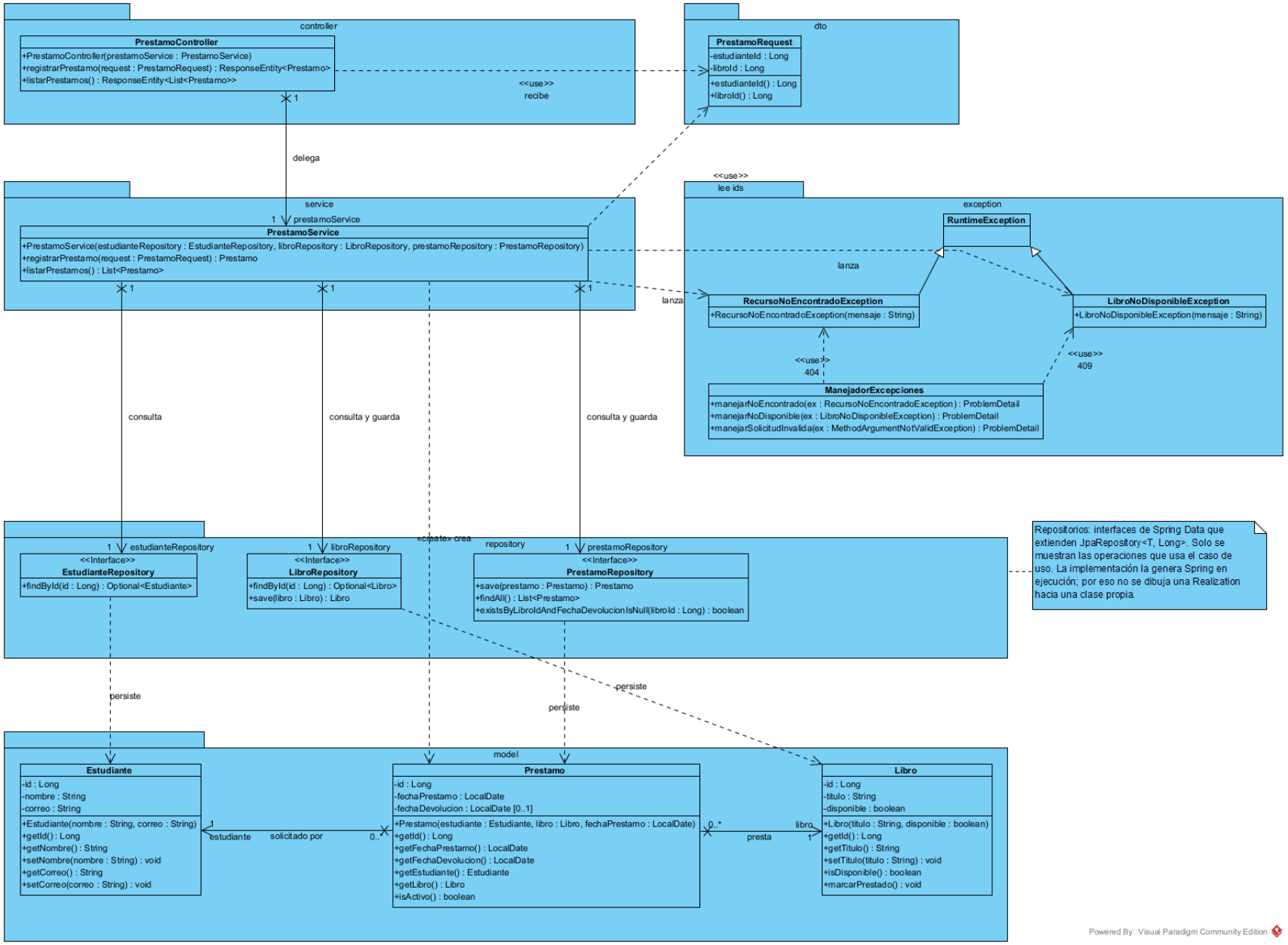


Figura 2. Modelo de clases por capas del caso de uso «Registrar préstamo» (diagramas/vpp/02-clases-por-capas.vpp).

4.1 Justificación de relaciones y multiplicidades

Relación	Tipo UML	Justificación
PrestamoController → PrestamoService	Asociación dirigida 1 → 1 (rol `prestamoService`)	El controlador conserva una referencia (atributo final inyectado por constructor); no es un uso puntual. Es navegable solo hacia el Service.
PrestamoService → Estudiante/Libro/PrestamoRepository	Asociaciones dirigidas 1 → 1	El Service mantiene una referencia a cada repositorio (inyección por constructor).
Prestamo → Estudiante	Asociación 0..* → 1, navegable hacia Estudiante (rol `estudiante`)	Cada préstamo tiene exactamente un estudiante y un estudiante puede tener cero o muchos préstamos. En JPA es `@ManyToOne`; Estudiante no guarda una lista.
Prestamo → Libro	Asociación 0..* → 1, navegable hacia Libro (rol `libro`)	Historial de préstamos por libro (R5). La condición de un solo préstamo activo no se puede expresar con una multiplicidad, así que la valida el Service.
PrestamoController ⇢ PrestamoRequest PrestamoService ⇢ PrestamoRequest	Dependencia «use»	Uso puntual como parámetro; no hay atributo ni referencia que se mantenga.
PrestamoService ⇢ Prestamo	Dependencia «create»	El Service instancia el préstamo (GRASP Creador) y no lo guarda como atributo.
PrestamoService ⇢ excepciones ManejadorExcepciones ⇢ excepciones	Dependencias (lanza / «use»)	El Service lanza las excepciones y el manejador las recibe como parámetro para producir 404 y 409.
Repositorios ⇢ entidades	Dependencia (persiste)	El repositorio usa el tipo de la entidad (`JpaRepository<T, Long>`), pero no conserva una referencia a ella.
Excepciones → RuntimeException	Generalización	Relación «es un»: ambas son excepciones no verificadas. La generalización no lleva multiplicidad.

Relaciones descartadas. No hay agregación ni composición, porque no existe una relación todo-parte. Tampoco se dibuja una realización: los repositorios son interfaces de Spring Data y su implementación la genera el framework en tiempo de ejecución; ninguna clase propia las implementa. Las interfaces se distinguen de las clases con el estereotipo «Interface».

5. Actividad 4 · Secuencia del caso de uso

El diagrama 03 muestra `POST /api/prestamos` con un fragmento **alt** de dos caminos: **préstamo exitoso** y **libro no disponible**. Además de los participantes mínimos se incluyen `libro:Libro`, para mostrar que el libro valida y cambia su propio estado, y `prestamo:Prestamo`, para mostrar su creación. Los caminos 404 siguen el mismo patrón y se describen en una nota del diagrama.

Diagrama 03 · Secuencia POST /api/prestamos

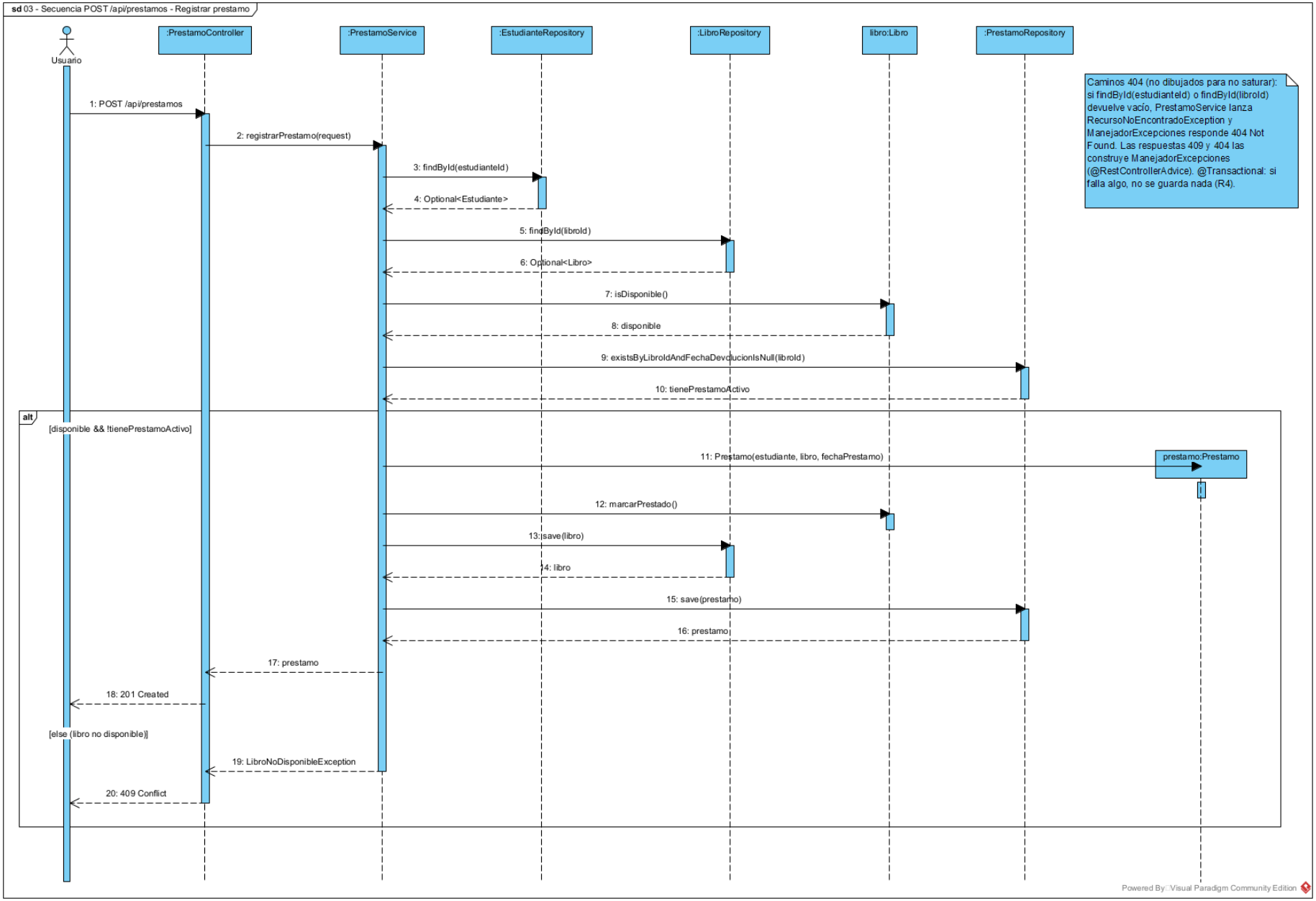


Figura 3. Secuencia del caso de uso «Registrar préstamo» (diagramas/vpp/03-secuencia-registrar-prestamo.vpp).

5.1 Verificación: cada mensaje corresponde a una operación del diagrama de clases

#	Mensaje en la secuencia	Operación en el diagrama de clases
1	POST /api/prestamos	`PrestamoController.registrarPrestamo(request: PrestamoRequest)`
2	registrarPrestamo(request)	`PrestamoService.registrarPrestamo(request: PrestamoRequest): Prestamo`
3	findById(estudianteId)	`EstudianteRepository.findById(id: Long): Optional<Estudiante>`
5 / 13	findById(libroId) / save(libro)	`LibroRepository.findById(id: Long)` / `save(libro: Libro): Libro`
7	isDisponible()	`Libro.isDisponible(): boolean`
9	existsByLibroIdAndFechaDevolucionIsNull(libroId)	`PrestamoRepository.existsByLibroIdAndFechaDevolucionIsNull(libroId: Long): boolean`
11	Prestamo(estudiante, libro, fechaPrestamo)	Constructor `Prestamo(estudiante, libro, fechaPrestamo)` (dependencia «create»)
12	marcarPrestado()	`Libro.marcarPrestado(): void`
15	save(prestamo)	`PrestamoRepository.save(prestamo: Prestamo): Prestamo`
18 / 20	201 Created / 409 Conflict	Retorno de `PrestamoController.registrarPrestamo` / `ManejadorExcepciones.manejarNoDisponible(ex)`

Los mensajes restantes son retornos (4, 6, 8, 10, 14, 16, 17 y 19).

6. Actividad 5 · Preparación de la implementación

Se creó un proyecto con Spring Initializr usando **Java 21, Maven, Spring Web, Spring Data JPA, Validation y H2** (Spring Boot 4.1.2). Los paquetes coinciden uno a uno con los del diagrama 02:

Paquete	Clases	Capa
controller	PrestamoController	Presentación
service	PrestamoService	Negocio
repository	EstudianteRepository, LibroRepository, PrestamoRepository	Acceso a datos
model	Estudiante, Libro, Prestamo	Modelo de dominio (entidades JPA)
dto	PrestamoRequest	Objeto de entrada
exception	RecursoNoEncontradoException, LibroNoDisponibleException, ManejadorExcepciones	Errores del caso de uso y su traducción a HTTP

Los datos iniciales están en `data.sql`: estudiantes 1 y 2, libros 1 y 2 disponibles, y el libro 3 con un préstamo activo. La consola de H2 está habilitada en `/h2-console`.

7. Actividad 6 · Implementación y pruebas

Los escenarios se verificaron de dos formas: con pruebas automáticas (`PrestamoControllerTest`, MockMvc sobre el contexto completo, ejecutadas con `mvnw test`) y con peticiones `curl` contra la aplicación en ejecución (`evidencias/pruebas-http.txt`).

Escenario	Resultado esperado	Resultado obtenido	Prueba automática
Préstamo de libro disponible	201 Created y préstamo registrado	201, Location `/api/prestamos/2`, libro con `disponible = false`	registrarPrestamoLibroDisponible ✓
Libro ya prestado	409 Conflict; sin nuevo préstamo	409, «El libro con id 1 no está disponible»; el número de préstamos no cambia	registrarPrestamoLibroNoDisponible ✓
Estudiante inexistente	404 Not Found	404, «No existe el estudiante con id 99»	registrarPrestamoEstudianteInexistente ✓
Libro inexistente	404 Not Found	404, «No existe el libro con id 99»	registrarPrestamoLibroInexistente ✓
Consulta de préstamos	200 OK con registros	200, lista JSON con los préstamos	listarPrestamos ✓
Extra: solicitud sin libroId (R4)	400 Bad Request; sin préstamo	400, «libroId es obligatorio»	registrarPrestamoSolicitudInvalida ✓

Resultado: 7 pruebas ejecutadas (6 escenarios más la prueba de arranque del contexto), 0 fallos y 0 errores. El detalle está en `evidencias/resultado-pruebas.txt`.
Decisiones que se reflejaron en UML durante la implementación: `@Transactional` en `registrarPrestamo`; las excepciones y `ManejadorExcepciones` (respuestas con `ProblemDetail`); la consulta `existsByLibroIdAndFechaDevolucionIsNull` para R5; y la eliminación de `setId` y `setDisponible`, porque el id lo genera la base de datos y la disponibilidad solo cambia con `marcarPrestado()`.

8. Actividad 7 · Revisión y defensa

8.1 ¿Qué responsabilidades cambiaron del modelo inicial al modelo por capas?

GestorPrestamos desaparece y sus responsabilidades se reparten: la coordinación de `registrarPrestamo` pasa a **PrestamoService**; las listas en memoria se reemplazan por los **repositorios** y H2; la recepción HTTP, que no existía, la asume **PrestamoController**; y los errores los traduce **ManejadorExcepciones**. En el dominio, Libro pierde `setDisponible()` y conserva `marcarPrestado()`; Prestamo gana `isActivo()`; los ids los genera la base de datos y por eso se eliminan sus setters.

8.2 ¿Qué patrón GRASP se reconoce en la coordinación? ¿Por qué un controlador GRASP no equivale necesariamente a @RestController?

Se reconoce el **Controlador GRASP** (controlador de caso de uso), que en este diseño es **PrestamoService**: es el primer objeto, más allá de la interfaz, que recibe el evento del sistema «registrar préstamo» y coordina a los demás. `@RestController` es otra cosa: un adaptador de la capa de presentación que traduce HTTP. Si se pusiera la coordinación en el `@RestController`, la lógica quedaría acoplada a HTTP y no podría reutilizarse desde otra interfaz (consola, mensajería o pruebas). Coinciden solo si el diseño concentra ambas responsabilidades, y eso es justamente lo que se quiere evitar. También aparecen **Experto** (Libro con su disponibilidad), **Creador** (el Service crea el Prestamo), **Alta cohesión** y **Bajo acoplamiento**.

8.3 ¿Dónde se ubica la regla de disponibilidad y por qué?

Tiene dos partes. El **estado** y su cambio están en **Libro** (`isDisponible()`, `marcarPrestado()`), porque es el experto en esa información. La **decisión** de rechazar el préstamo está en **PrestamoService**, porque combina datos de varias fuentes (el libro y los préstamos activos del repositorio) y es una regla del caso de uso. No va en el Controller, que mezclaría negocio con HTTP, ni en el Repository, que solo persiste.

8.4 ¿Qué relaciones UML conservaron y cuáles modificaron?

- **Conservadas:** las asociaciones Estudiante–Prestamo y Libro–Prestamo con multiplicidad 1 / 0..*, sin agregación ni composición.
- **Modificadas:** ahora son navegables solo desde Prestamo (`@ManyToOne`) y los atributos `estudiante` y `libro` se representan como roles de la asociación, en lugar de repetirse como atributos.
- **Eliminadas:** las tres asociaciones de GestorPrestamos (registra, administra y gestiona).
- **Nuevas:** asociaciones dirigidas Controller → Service → Repository; dependencias «use» y «create»; dependencias de persistencia; generalización de las excepciones.

8.5 ¿Cómo se comprueba la trazabilidad requisito → clase → operación → prueba?

Con la matriz siguiente. Cada regla apunta a la clase y la operación que la implementan (que aparecen en los diagramas 02 y 03) y a la prueba automática que la verifica:

Regla	Clase	Operación	Prueba
R1	Libro / PrestamoService	`isDisponible()` / `registrarPrestamo()`	registrarPrestamoLibroNoDisponible (409)
R2	Prestamo / PrestamoService	`Prestamo(estudiante, libro, fechaPrestamo)`	registrarPrestamoLibroDisponible (verifica estudiante, libro y fecha)
R3	Libro	`marcarPrestado()`	registrarPrestamoLibroDisponible (`disponible = false`)

Regla	Clase	Operación	Prueba
R4	PrestamoRequest / PrestamoController / ManejadorExcepciones / PrestamoService	`@NotNull`, `@Valid`, `manejarSolicitudInvalida()`, `@Transactional`	registrarPrestamoSolicitudInvalida (400); escenarios 404 sin cambios en BD
R5	PrestamoRepository / Prestamo	`existsByLibroIdAndFechaDevolucionIsNull()` / `isActivo()`	registrarPrestamoLibroNoDisponible (libro con préstamo activo); listarPrestamos (`activo = true`)

9. Decisiones de diseño

1. **Tres capas más un modelo de dominio.** Controller (HTTP), Service (caso de uso y reglas) y Repository (persistencia), con dependencias en una sola dirección. Las entidades son compartidas y no forman una cuarta capa. Esto da alta cohesión por clase y permite cambiar la persistencia o la interfaz sin tocar las reglas.
2. **La regla vive donde está la información.** Libro controla su disponibilidad (Experto) y el Service decide el préstamo (Controlador de caso de uso). Se elimina `setDisponible()` para que nadie pueda saltarse R1 y R3.
3. **DTO de entrada con identificadores.** `PrestamoRequest(estudianteId, libroId)` desacopla el contrato HTTP de las entidades y permite validar el formato con `@NotNull` antes de llegar al negocio (R4).
4. **Préstamo activo = fechaDevolucion vacía.** Se reutiliza el atributo `[0..1]` del modelo inicial en lugar de añadir un estado nuevo. El Service comprueba R5 con una consulta al repositorio, además de la bandera `disponible`, para no depender de un solo dato.
5. **Transacción.** `registrarPrestamo` es `@Transactional`: el libro marcado como prestado y el préstamo nuevo se confirman juntos o se revierten juntos. La concurrencia (dos solicitudes simultáneas por el mismo libro) podría reforzarse más adelante con bloqueo optimista (`@Version`).
6. **Errores explícitos.** Las excepciones del caso de uso (`RecursoNoEncontradoException` y `LibroNoDisponibleException`) se traducen en un solo lugar, `ManejadorExcepciones` (`@RestControllerAdvice`), a 404, 409 y 400 con el formato estándar `ProblemDetail`. El Service no conoce HTTP.
7. **Repositorios como interfaces de Spring Data.** Se modelan con el estereotipo «Interface» y solo con las operaciones que usa el caso de uso. No se dibuja realización porque la implementación la genera el framework.
8. **Simplificaciones conscientes.** La respuesta devuelve la entidad Prestamo; una mejora sería un `PrestamoResponse`. Las reglas de la guía del curso que no pide este taller (ejemplares, máximo tres préstamos activos, fecha límite) se dejan para iteraciones posteriores.